

Session 4 · Deck 4.A1

Root-Finding: Opening the Black Box

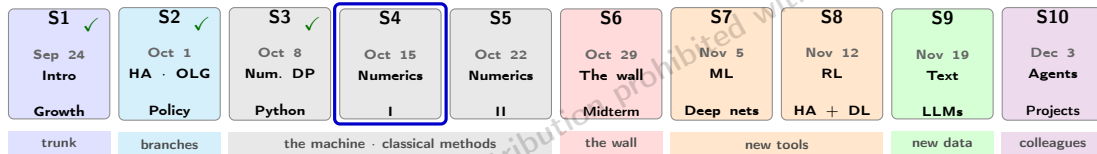
Zhigang Feng

Thursday, October 15, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 4.A1 ▼



Build it on paper → solve it on the machine → push it to the frontier with AI

Block A: 4.A1 root-finding · 4.A2 optimization on an interval · 4.A3 optimization in many dimensions · 4.A4 differentiation.

Five Methods, One Equation, and No Solution

A market-clearing problem, solved five ways in a notebook:

$$Q^d(p) = 100 - 2p + 10\sqrt{p}, \quad Q^s(p) = -20 + 3p - 0.1p^2.$$

bisection	ValueError
Brent	ValueError
Newton	a number
secant	a number

Excess demand has a **minimum of** +104.7 at $p = 19.3$ and is positive everywhere. There is no equilibrium price. The two open methods return one anyway — *which* number depends on the implementation, and at every one of them excess demand still exceeds 100.

Which two behaved well?

The ones that **refused**. Bisection and Brent need a bracket with a sign change; when you cannot supply one, they tell you so.

That is the whole lesson of this deck in one slide. A root-finder does not know what a price is. It reports a number; **whether the number means anything is your job**, and the methods differ in how loudly they fail.

Why Economists Spend So Much Time Solving $f(x) = 0$

- **Market clearing:** excess demand = 0 — the price r^* in Aiyagari is exactly this.
- **First-order conditions:** the labour FOC $u_\ell = w u_c$, solved at every state.
- **Euler equations:** 3.A3's time iteration solves one root per grid point per sweep.
- **Any fixed point:** $x = g(x)$ becomes $f(x) = x - g(x) = 0$.

The last line matters: **every fixed-point problem in this course is a root-finding problem**, including the ones we have been solving by iteration.

Two families

Bracketing methods keep an interval known to contain a root. Slow, and they cannot fail.

Open methods follow a local model of f . Fast, and they can run away.

`brentq`, which 3.A3 used as a sealed box, is a bracketing method wrapped around an open one — which is why it is the default.

Motivation and the method taxonomy adapted from QM *Numerical Methods, Research Architect*, Part III.

Bisection: The One That Cannot Fail

Given $[a, b]$ with $f(a)f(b) < 0$ and f continuous, the intermediate value theorem puts a root inside. Halve, keep the half that still changes sign, repeat.

$$|x_n - x^*| \leq \frac{b - a}{2^n}.$$

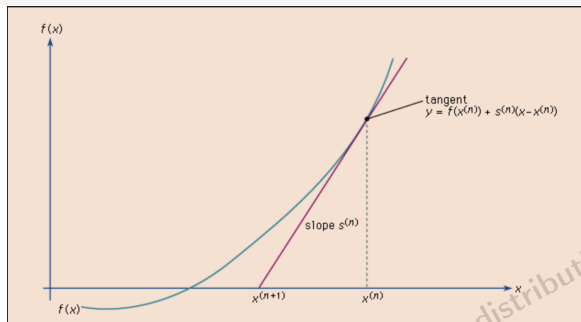
- **Linear** convergence: one bit per iteration, about 3.3 iterations per decimal digit.
- The bound is *a priori* — you know the iteration count before you start, from the bracket alone.
- It uses nothing about f beyond its sign.

Why it is still the right default

It is the only method here whose guarantee does not depend on where you started, on differentiability, or on luck.

Its one requirement is the bracket, and finding one is **your** problem — as the hook showed. Plot the function first.

Newton: Fast, and Only Locally Safe



$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}, \quad e_{n+1} \approx \frac{f''}{2f'} e_n^2,$$

so the error is **squared** each step — the number of correct digits doubles. Cost: one f and one f' per iteration.

Figure newton.png and the quadratic-convergence discussion from QM *Numerical Methods*, Newton block, copied in.

Three ways it goes wrong

$f'(x_n) \approx 0$ sends the next iterate to infinity · a bad start converges to a different root, or to none · it will happily leave the economically sensible region, as it did on the hook slide.

The standard repair is **damping**: take $x_{n+1} = x_n - \theta f/f'$ with $\theta \in (0, 1]$, backtracking on θ until $|f|$ actually falls.

Secant and Brent: Newton Without the Derivative

Secant. Replace $f'(x_n)$ by the slope through the last two points:

$$x_{n+1} = x_n - f(x_n) \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}.$$

One new evaluation per step, no derivative, and the order is the golden ratio $\frac{1+\sqrt{5}}{2} \approx 1.618$ — slower than Newton per step, often faster per *second*, because f' is usually the expensive part.

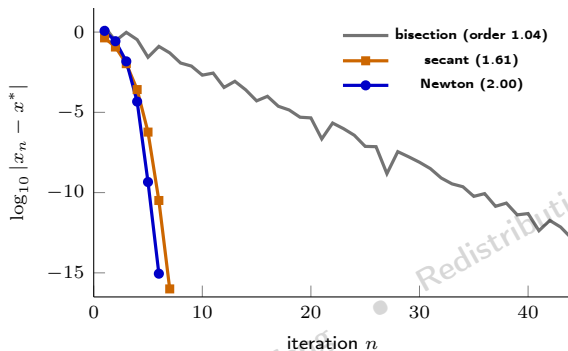
Brent. Keep a bracket, and inside it use inverse quadratic interpolation when that helps and bisection when it does not. Superlinear when the function is nice; never worse than bisection when it is not.

What was in the box

This is brentq. 3.A3 was entitled to trust it because we had *proved* the residual crosses zero exactly once — and a bracketing method with a valid bracket cannot fail.

The general rule: **bracket if you can, and only then go fast.** Every production solver is built this way.

The Rates, Measured



Market clearing for $100p^{-0.5} = e^{0.5p} - 1$, whose root is $p^* = 7.2786$.

	to 10^{-12}	order
bisection	41	1.04
secant	7	1.61
Newton	6	2.00

The observed orders are 1.04, 1.61 and 2.00 against theoretical 1, 1.618 and 2. The theory is not decoration — it predicts the slopes on the left to two digits.

Observed order estimated from the last clean error triple, $\log(e_{n+1}/e_n)/\log(e_n/e_{n-1})$. Bisection's error is not monotone, so that estimator is noisy for it; its rate is linear with ratio $1/2$ per step, which is the cleaner statement.

Systems: The Jacobian, and Why You Avoid Rebuilding It

For $F : \mathbb{R}^N \rightarrow \mathbb{R}^N$, Newton becomes

$$J(x_n) \Delta_n = -F(x_n), \quad x_{n+1} = x_n + \Delta_n,$$

with $J_{ij} = \partial F_i / \partial x_j$. Two costs, and the second is the one that hurts:

- **building** J : N^2 partial derivatives, each needing an evaluation if done by finite differences;
- **solving** the linear system: $O(N^3)$.

Broyden updates an *approximate* J from the step it just took, so it builds a Jacobian once and never again.

$x^2 + y^2 = 4, e^x + y = 1$	iters	Jacobians
Newton	5	5
Broyden	14	1

Both reach a residual of 0 to machine precision. Broyden takes nearly three times the iterations and one fifth of the derivative work — and when J is the expensive part, that is the trade you want.

Cost accounting adapted from QM *Numerical Methods*, “Why Multivariate is Hard”. Where the derivatives come from is 4.A4.

Fixed Points, and the Cheapest Repair in Numerical Economics

Many equilibrium problems arrive as $x = g(x)$ and are solved by just iterating it. That converges only if $|g'(x^*)| < 1$, and in economics it frequently is not.

Damping (relaxation) fixes it in one line:

$$x_{n+1} = (1 - \theta)x_n + \theta g(x_n), \quad \theta \in (0, 1].$$

The map's slope becomes $1 - \theta + \theta g'$, so a small enough θ always pulls it inside the unit circle.

$g(x) = 4 - x^2/4$	iterations
$\theta = 1$ (no damping)	diverges
$\theta = 0.5$	11
$\theta = 0.2$	41

Undamped iteration never converges here. At $\theta = 0.5$ it takes eleven steps. **Too small a θ is also a cost** — 0.2 needs four times as many.

This is exactly how the outer loop on r in Aiyagari is stabilised (S5).

Four Checks Before You Believe a Root

1. **Did it converge?** Read the solver's own flag — do not assume. Both failures on the hook slide reported a number.
2. **Is the residual small?** $|f(x)| < \text{tol}$, checked by you, in the units of the problem.
3. **Is it feasible?** A negative price or a negative consumption is not a solution, whatever the residual says.
4. **Is it unique?** Re-solve from twenty different starts and count the distinct answers.

The failure mode to know

The notebook on the hook slide printed a ✓ next to both wrong answers, because the tick was hard-coded rather than read off `result.converged`. It then *validated* $p = 245$ as the equilibrium.

Check 4 is the one people skip, and it is the one that matters in economics: **multiple equilibria are a result, not a bug** — but only if you looked.

The four-check validator is adapted from QM *Numerical Methods, Research Architect*, Part III.

Takeaways

- Every fixed point in this course is a root-finding problem, and brentq is no longer a black box: it is **a bracket with a fast method inside it.**
- The rates are real and we measured them — observed orders 1.04, 1.61 and 2.00 against theoretical 1, 1.618 and 2. Newton reached 10^{-12} in **six** iterations; bisection took 41.
- Speed and safety trade off exactly as the theory says. **Bracket if you can**, then go fast inside the bracket.
- In systems, the Jacobian is the expense. Broyden pays three times the iterations for one fifth of the derivative work.
- A solver that returns a number has not told you it is right. **Read the convergence flag, check the residual, check feasibility, and look for other roots.**

Bridge: The Other Half of the Inner Loop

Time iteration needed a root per grid point, and we have now opened that box. But 3.A2 and 3.A4 had a different inner loop: a **maximization**, done by searching all N candidates on the grid — and it left a staircase in the policy that no amount of iterating removed.

Maximizing is not the same problem as root-finding, and the safe-versus-fast trade-off comes back in a different shape: without derivatives, the analogue of bisection is not halving an interval but shrinking it by a very particular ratio.

Next (4.A2)

Optimization on an interval: golden section, why the ratio is golden, and Brent's minimizer.

Then the pay-off — the 3.A4 staircase retired, and what continuous choice actually bought us.